

5. Data model

- Attributs spéciaux
- Méthodes spéciales
- opérateurs

Attributs spéciaux

Les attributs spéciaux (entourés de double underscore) sont accessibles depuis le type d'une classe ou l'instance d'une classe. Ceux-ci sont en lecture seule.

Nom	Accessible sur	Objectif
<code>__name__</code>	classe	Renvoie le nom de la classe.
<code>__bases__</code>	classe	Renvoie un tuple des classes parentes de la classe.
<code>__class__</code>	objet	Permet de connaître à partir de quelle classe l'objet a été créé.
<code>__dict__</code>	objet	Renvoie un dictionnaire des méthodes et attributs disponibles.
<code>__doc__</code>	classe et objet	Renvoie la “docstring” de la classe interrogée.

Attributs spéciaux : démonstration

```
1 class MaClasse:
2     """Ceci est la documentation de MaClasse."""
3
4     def methode(self):
5         pass
6
7     # Accéder à l'attribut __name__
8     print("Nom de la classe:", MaClasse.__name__)
9
10    # Accéder à l'attribut __bases__
11    print("Classes parentes:", MaClasse.__bases__)
12
13    # Créer un objet de la classe MaClasse
14    objet = MaClasse()
15
16    # Accéder à l'attribut __class__
17    print("Classe de l'objet:", objet.__class__)
18
19    # Accéder à l'attribut __dict__
20    print("Méthodes et attributs disponibles:", objet.__dict__)
21
22    # Accéder à l'attribut __doc__
23    print("Documentation de la classe:", MaClasse.__doc__)
```

Méthodes spéciales

Les méthodes spéciales sont des fonctions prédéfinies avec des noms spéciaux entourés de double underscore également permettent de définir le comportement d'une classe. Celle-ci peuvent être redéfinies par la classe.

Nom	Rôle
<code>__init__</code>	Permet l'initialisation des champs corrects lors de la création de l'objet.
<code>__repr__</code>	Envoie la représentation "officielle" en chaîne de caractères d'un objet.
<code>__str__</code>	Envoie la représentation "informelle" en chaîne de caractères d'un objet.
<code>__len__</code>	Renvoie la longueur de l'objet.
<code>__getitem__</code>	Permet d'accéder à un élément de l'objet en utilisant la notation d'indexation (par exemple, objet[key]).

Méthodes spéciales : démonstration

```
1 class Voiture:
2     _marque = "Honda"
3     _model = "Civic"
4     _annee = 2006
5
6     def __repr__(self):
7         return f"voiture({self._marque}, {self._model}, {self._annee})"
8
9     def __str__(self):
10        return f"{self._annee} {self._marque} {self._model}"
11
12    def __len__(self):
13        # Définir la longueur de la voiture
14        return len(str(self))
15
16    def __getitem__(self, key):
17        # Permet d'accéder à un attribut spécifique de la voiture
18        if key == 'marque':
19            return self._marque
20        elif key == 'model':
21            return self._model
22        elif key == 'annee':
23            return self._annee
24        else:
25            raise KeyError(f"Attribut '{key}' non valide pour la voiture")
```

```
27 # Création d'une voiture
28 voiture = Voiture()
29
30 # Affichage de la représentation officielle et informelle de la voiture
31 print(repr(voiture)) # Affiche la représentation officielle
32 print(str(voiture))  # Affiche la représentation informelle
33
34 # Obtention de la longueur de la voiture
35 print(len(voiture))
36
37 # Accès aux attributs de la voiture en utilisant la notation d'indexation
38 print(voiture['marque'])
39 print(voiture['model'])
40 print(voiture['annee'])
```

Opérateurs spéciaux

Il sera possible également de manipuler les opérateurs afin de redéfinir le comportement lors de la comparaison.

Méthode spéciale	Rôle	code classique
<code>__eq__</code>	Renvoie True si les objets sont égaux, False sinon.	<code>obj1 == obj2</code>
<code>__lt__</code>	Renvoie True si l'objet actuel est strictement inférieur à l'autre objet, False sinon.	<code>obj1 < obj2</code>
<code>__le__</code>	Renvoie True si l'objet actuel est inférieur ou égal à l'autre objet, False sinon.	<code>obj1 <= obj2</code>
<code>__gt__</code>	Renvoie True si l'objet actuel est strictement supérieur à l'autre objet, False sinon.	<code>obj1 > obj2</code>
<code>__ge__</code>	Renvoie True si l'objet actuel est supérieur ou égal à l'autre objet, False sinon.	<code>obj1 >= obj2</code>

Opérateurs spéciaux : démonstration

```
1 class Car:
2     _marque = ""
3     _model = ""
4     _annee = 0
5     ...
6     def __eq__(self, other): # Opérateur ==
7         return self._marque == other._marque and self._model == other._model and self._annee == other._annee
8     ...
9     def __ne__(self, other): # Opérateur !=
10        return not self == other
11    ...
12    def __lt__(self, other): # Opérateur <
13        if self._annee != other._annee:
14            return self._annee < other._annee
15        elif self._marque != other._marque:
16            return self._marque < other._marque
17        else:
18            return self._model < other._model
19    ...
20    def __le__(self, other): # Opérateur <=
21        return self < other or self == other
22    ...
23    def __gt__(self, other): # Opérateur >
24        return not (self < other or self == other)
25    ...
26    def __ge__(self, other): # Opérateur >=
27        return not (self < other)
```

```
29 # Création de quelques voitures
30 car1 = Car()
31 car1._marque = "Toyota"
32 car1._model = "Camry"
33 car1._annee = 2020
34
35 car2 = Car()
36 car2._marque = "Honda"
37 car2._model = "Accord"
38 car2._annee = 2019
39
40 car3 = Car()
41 car3._marque = "Toyota"
42 car3._model = "Corolla"
43 car3._annee = 2020
44
45 # Comparaison des voitures
46 print(car1 == car2) # False
47 print(car1 != car2) # True
48 print(car1 < car2) # True
49 print(car1 > car2) # False
50 print(car1 <= car3) # True
51 print(car2 >= car3) # False
```

6. Heritage

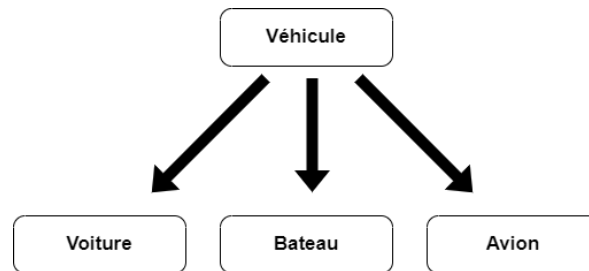
- Définition
- Héritage simple
- La méthode `<< Super() >>`
- Héritage multiple
- Redéfinition de méthodes
- Trucs et astuces

Définition de l'héritage

L'héritage est un concept fondamental en programmation orientée objet, offrant la possibilité aux classes de partager des attributs et des méthodes avec d'autres classes, appelées superclasses.

Cette relation parent-enfant permet aux sous-classes d'hériter et d'étendre les fonctionnalités de la superclasse, favorisant ainsi la réutilisabilité du code et la modularité de la conception logicielle.

En héritant des caractéristiques de la superclasse, les sous-classes peuvent bénéficier d'une implémentation prédéfinie, tout en ayant la flexibilité d'ajouter leurs propres fonctionnalités spécifiques.



Heritage simple

Pour mettre en place l'héritage, nous devons suivre une structure claire et méthodique. Tout d'abord, nous définissons la classe parente, qui contient les attributs et les méthodes communs à plusieurs classes.

Ensuite, nous déclarons la classe enfant en spécifiant entre parenthèses le nom de la classe parente, indiquant ainsi l'héritage. Dans la sous-classe, nous pouvons personnaliser le comportement en ajoutant de nouveaux attributs ou en définissant des nouvelles méthodes.

```
1 class Vehicule:
2     marque = "honda"
3     model = "Civic"
4     def afficher_info(self):
5         print("Marque :", self.marque)
6         print("Modèle :", self.model)
7
8
9 class Voiture(Vehicule):
10     couleur = None
11
12
13 # Création d'une instance de la classe Voiture
14 ma_voiture = Voiture()
15 ma_voiture.marque = "Toyota"
16 ma_voiture.model = "Corolla"
17 ma_voiture.couleur = "rouge"
18
19 # Affichage des informations de la voiture
20 print("Informations de la voiture :")
21 ma_voiture.afficher_info()
```

La méthode << Super() >>

La méthode `super()` permet d'accéder aux méthodes et aux attributs de la classe parente à partir de la classe enfant.

Cela est particulièrement utile lorsque nous redéfinissons des méthodes dans la sous-classe tout en souhaitant appeler la méthode équivalente de la classe parente.

`super()` permet donc d'appeler la méthode parente de manière dynamique et flexible, sans avoir à spécifier explicitement le nom de la classe parente.

```
9  class Voiture(Vehicule):
10      couleur = "Rouge"
11
12      def afficher_info(self):
13          super().afficher_info()
14          print("Couleur :", self._couleur)
```

Heritage multiple

En Python, contrairement à d'autres langages, il est possible d'appliquer l'héritage multiple. Ce qui signifie qu'une classe peut hériter des attributs et des méthodes de plusieurs autres classes.

Cela permet à une classe enfant d'avoir accès aux fonctionnalités de plusieurs classes parentes.

```
1 class Vehicule:
2     marque = None
3
4     def afficher_marque(self):
5         print("Marque du véhicule :", self.marque)
6
7
8 class Modele:
9     model = None
10
11     def afficher_modele(self):
12         print("Modèle du véhicule :", self.model)
13
14
15 class Voiture(Vehicule, Modele):
16
17     def afficher_info(self):
18         self.afficher_marque()
19         self.afficher_modele()
20
21
22 # Création d'une instance de la classe Voiture
23 ma_voiture = Voiture()
24 ma_voiture.marque = "Toyota"
25 ma_voiture.model = "Corolla"
26
27 # Affichage des informations de la voiture
28 ma_voiture.afficher_info()
```

Redéfinition de méthodes

La redéfinition de méthode permet de redéfinir une méthode existante dans la classe parente dans une classe enfant, en fournissant une implémentation spécifique à cette classe enfant.

Cela permet à la classe enfant de modifier ou d'étendre le comportement de la méthode héritée de la classe parente sans modifier la classe parente elle-même.

Cette technique est utile pour personnaliser le comportement des méthodes en fonction des besoins spécifiques de chaque classe enfant.

```
1 class Vehicule:
2     def demarrer(self):
3         print("Le véhicule démarre.")
4
5     def arreter(self):
6         print("Le véhicule s'arrête.")
7
8
9 class Voiture(Vehicule):
10     def demarrer(self):
11         print("La voiture démarre en appuyant sur l'accélérateur.")
12
13     # La méthode arreter() n'est pas redéfinie, donc elle est héritée de la classe parente
14
15
16 # Création d'une instance de la classe Voiture
17 ma_voiture = Voiture()
18
19 # Appel des méthodes surchargées
20 ma_voiture.demarrer() # Affiche: La voiture démarre en appuyant sur l'accélérateur.
21 ma_voiture.arreter()  # Affiche: Le véhicule s'arrête.
```

Trucs et astuces : `issubclass()`

La fonction `issubclass()` permet de vérifier si une classe est une sous-classe d'une autre classe. Elle prend deux arguments : la classe enfant et la classe parente, et renvoie `True` si la première est une sous-classe de la seconde, sinon `False`.

Cette fonction est utile pour vérifier les relations d'héritage entre les classes et peut être utilisée dans divers contextes, comme la gestion des exceptions ou la définition de comportements spécifiques basés sur la hiérarchie des classes.

```
1  class Vehicule:
2      pass
3
4  class Voiture(Vehicule):
5      pass
6
7  class Moto(Vehicule):
8      pass
9
10 # Vérification si Voiture est une sous-classe de Vehicule
11 print(issubclass(Voiture, Vehicule)) # Output: True
12
13 # Vérification si Moto est une sous-classe de Vehicule
14 print(issubclass(Moto, Vehicule))    # Output: True
15
16 # Vérification si Vehicule est une sous-classe de Moto
17 print(issubclass(Vehicule, Moto))    # Output: False
```

Exercices

1. Créez de la classe Girafe qui reprendra les mêmes éléments que la classe Éléphant mais qui implémentera maintenant en plus les éléments suivants :

Caractéristiques :

- longueur_cou (lecture & écriture)

Comportement :

- manger_feuilles()
- boire_eau()

Exercices

2. Adaptez la classe Éléphant afin qu'elle contienne à présent les éléments suivants :

Caractéristiques :

- longueur_defense (lecture & ecriture)

Comportement :

- prendre_bain_de_boue()
- aspirer_eau()

Exercices

3. Mettez en place le concept d'héritage en créant la classe `Animal` qui reprendra les éléments communs aux classes `Girafe` et `Éléphant`.
4. Définissez une méthode `observer_environnements()` au sein de cette classe `Animal` et redéfinissez-la ensuite dans les enfants pour adapter son implémentation.
5. Réalisez des tests en implémentant les différentes fonctionnalités ajoutées et vérifiez leurs bon fonctionnement.